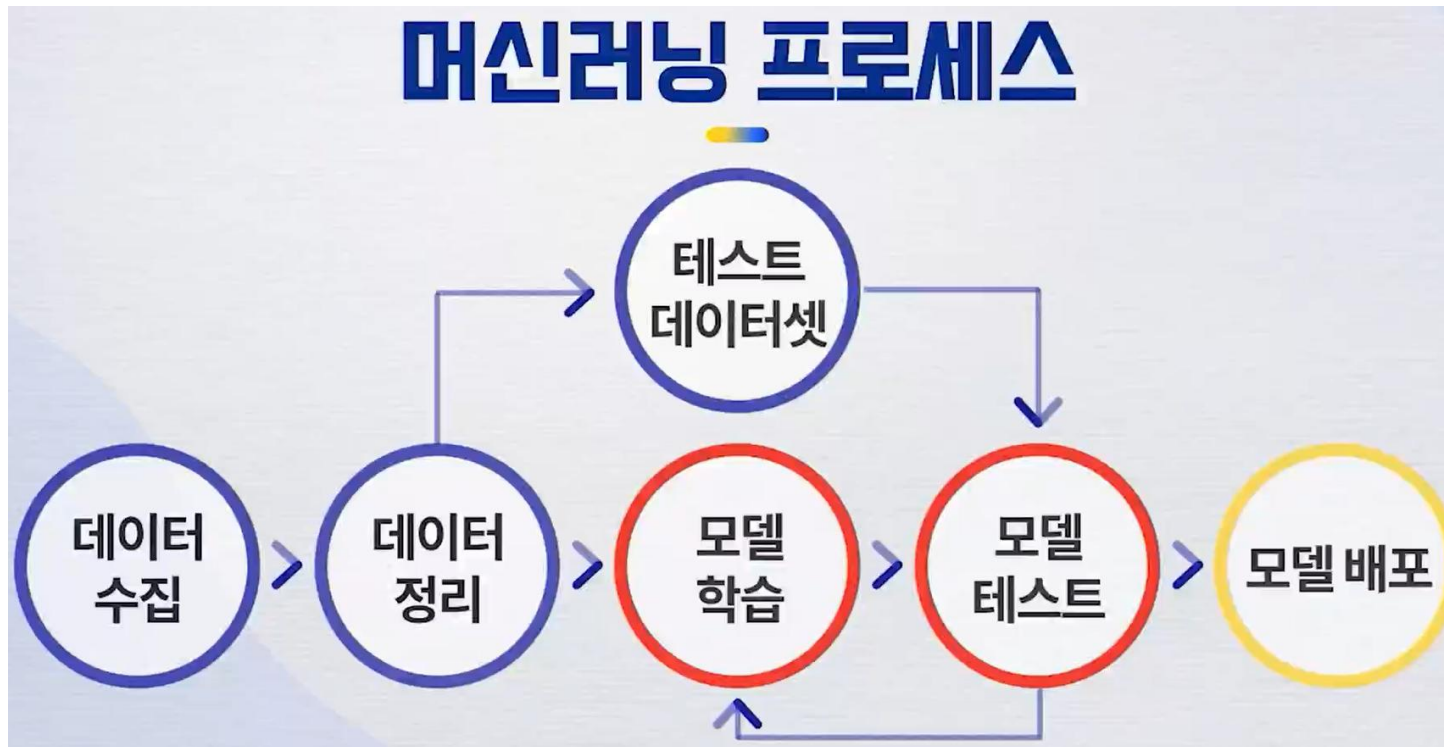


머신러닝

- 머신러닝 기본 개념
- 머신러닝 기술원리
- 머신러닝 주요 알고리즘
- 파이썬 실습

머신러닝(Machine Learning, 기계학습)



머신러닝 개념(2/10)

과거

Data

Model

Prediction



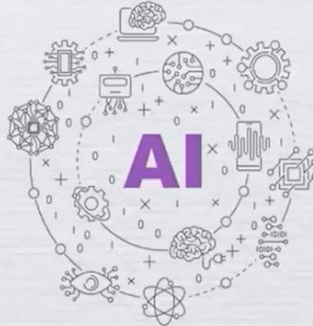
▶ 사람이 직접 알고리즘을 코딩하거나 패턴을 만들어 결과를 얻음

AI/머신러닝

Data

Model

Prediction



▶ 데이터와 결과를 기반으로 스스로 패턴 학습하고 이를 이용해서 예측

모델(model)은 원래 "어떤 것을 단순화하거나 축소해서 표현한 표본·형태"라는 뜻
예: 건물 축소 모형, 패션모델, 수학적 모델 등

원본(현실)을 완벽히 복제하지 않고 본질적 특징만 추출하여 표현

머신러닝에서 '모델'의 의미

훈련데이터(Training Data)는 현실 세계의 일부

학습(Training)은 데이터에서 규칙·패턴·관계를 찾아내는 과정

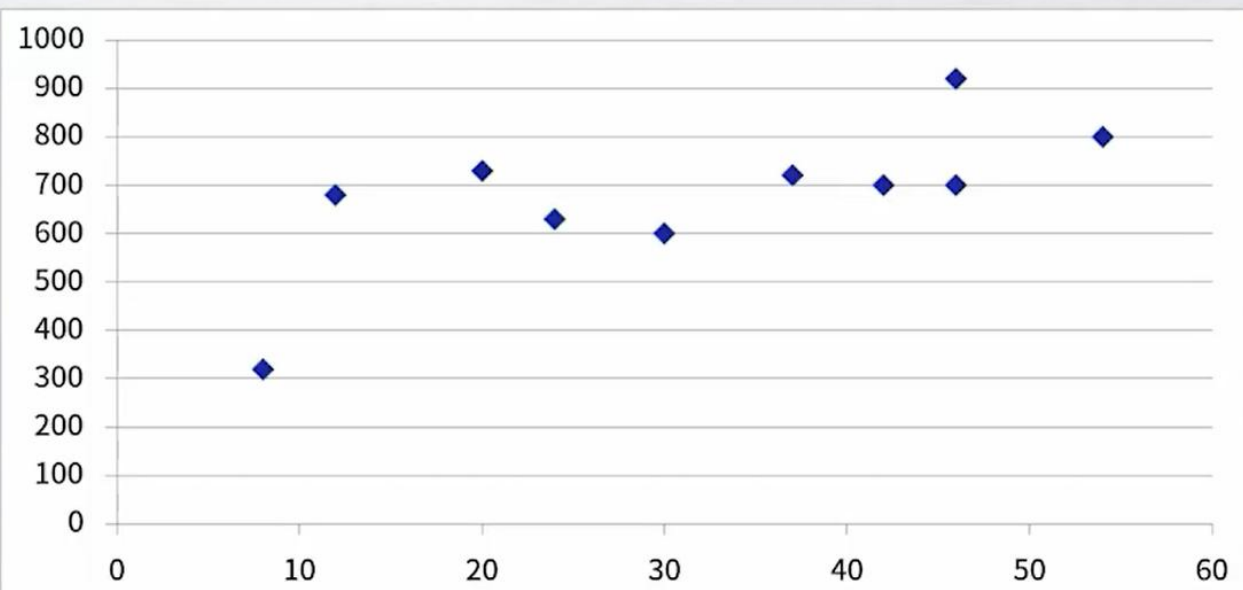
학습이 끝난 후 만들어진 수학적 구조나 파라미터 집합이 바로 모델

- 예: 선형회귀 → $y = ax + b$ 에서 a 와 b 값이 모델의 핵심
- 예: 신경망 → 수많은 가중치(weight)와 편향(bias)의 집합

Linear Regression

가설 : “공부를 많이 하면 공부를 잘할 것이다.”

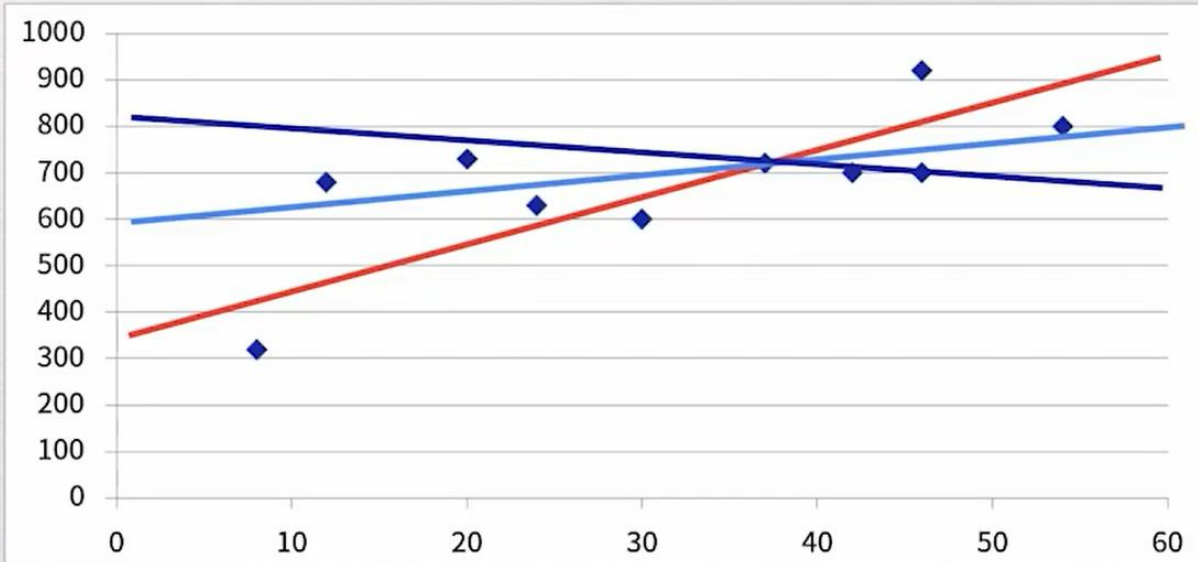
학습시간(시간)	토익성적(점)
54	800
8	320
30	600
24	630
46	700
12	680
20	730
37	720
42	700
46	920



Linear Regression

우리의 목표는?

학습시간(시간)	토익성적(점)
54	800
8	320
30	600
24	630
46	700
12	680
20	730
37	720
42	700
46	920



회귀모델
회귀식

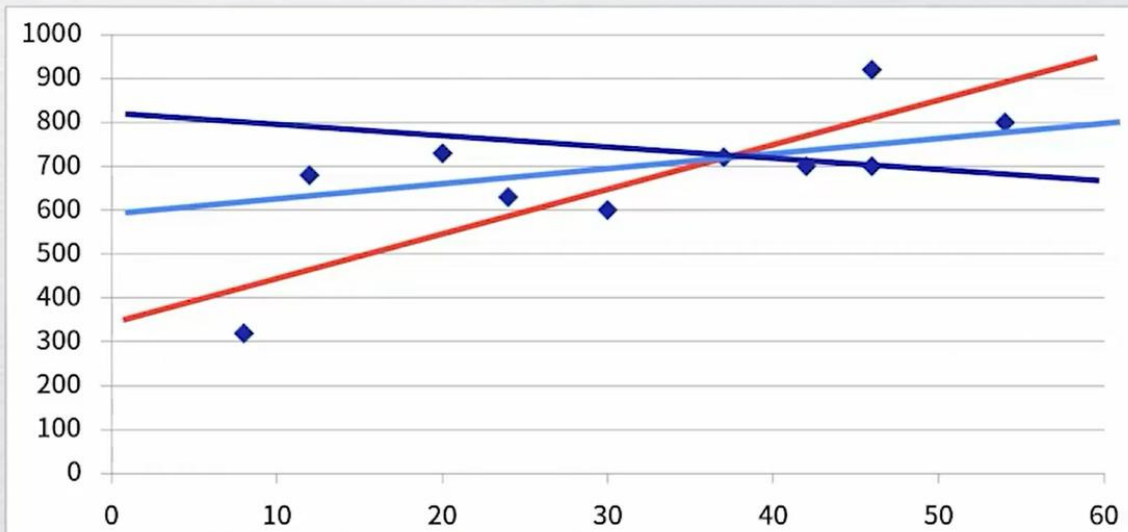
$$y = ax + b$$

점들의 분포패턴에
가장 잘 맞는 직선 찾아야
(최적의 a와 b)

Linear Regression

가장 잘 맞는 직선을 구한다는 의미는?

학습시간(시간)	토익성적(점)
54	800
8	320
30	600
24	630
46	700
12	680
20	730
37	720
42	700
46	920



$$y = wx + b$$

단순화

$$y = wx$$

y : 토익성적

x : 학습시간

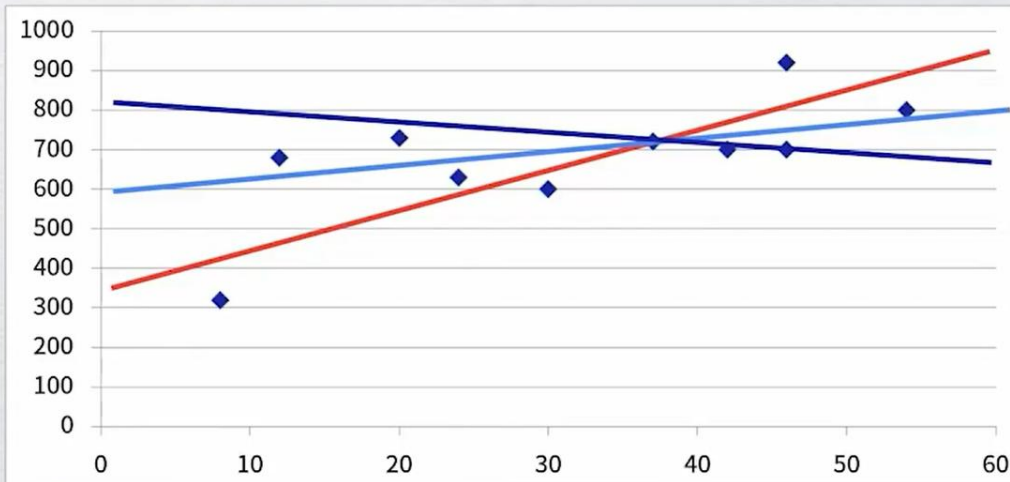
w: 가중치(기울기, 회귀계수)

많은 후보 w중에서
점들의 분포패턴에 가장 잘 맞는
w를 찾아야 함
-> 빨간색 직선에 해당하는 w를 찾아야.

Linear Regression

컴퓨터는 W 가중치가 최적인지 어떻게 알 수 있을까?

학습시간(시간)	토익성적(점)
54	800
8	320
30	600
24	630
46	700
12	680
20	730
37	720
42	700
46	920

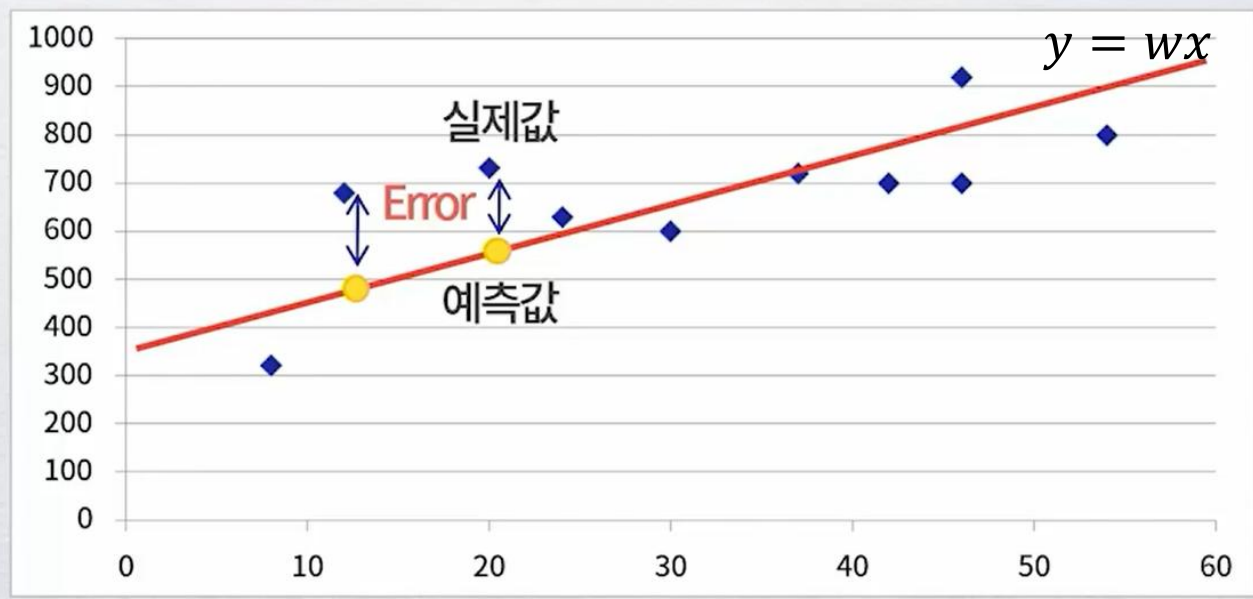


많은 후보 w 중에서
점들의 분포패턴에 가장 잘 맞는
 w 를 찾아야 함
→ 빨간색 직선에 해당하는 w 찾음

MSE(Mean Squared Error, 평균제곱오차)
를 최소화하는 w 를 찾음

비용함수(**Cost Function**)를 최소로 하는
 w 를 구함

Linear Regression



예측값 - 실제값 = Error, cost

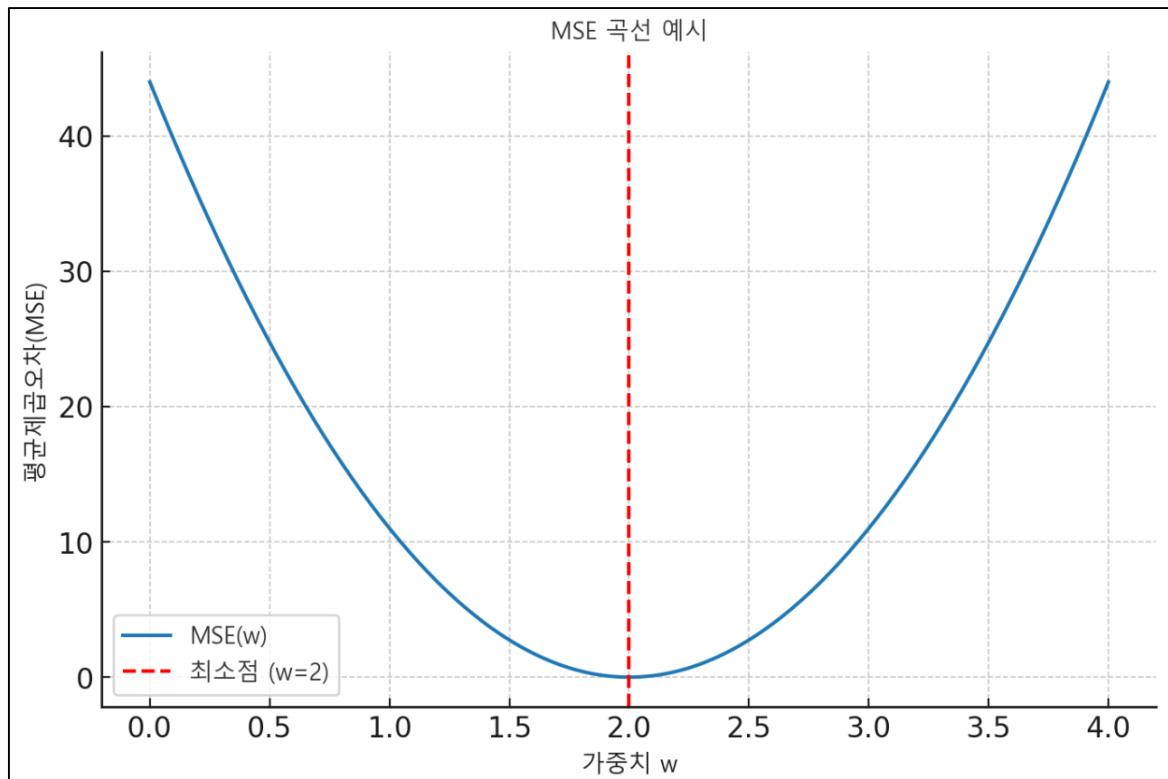
예측값과 실제값의 차이를 제곱하여 모두 더한 뒤, 데이터 개수로 나눈 값이 MSE
(**MSE**는 Linear Regression의 **비용함수**)

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

- n : 데이터의 개수
- y_i : 실제값 (Actual value)
- $\hat{y}_i$: 예측값 (Predicted value)
- $(\hat{y}_i - y_i)^2$: 예측값과 실제값의 차이를 제곱한 값
- 평균을 내기 위해 $\frac{1}{n}$ 을 곱함

머신러닝 개념(8/10)

MSE 비용함수의 2차원 그래프 모양(



$$\text{MSE}(w) = \frac{1}{n} \sum_{i=1}^n (wx_i - y_i)^2$$

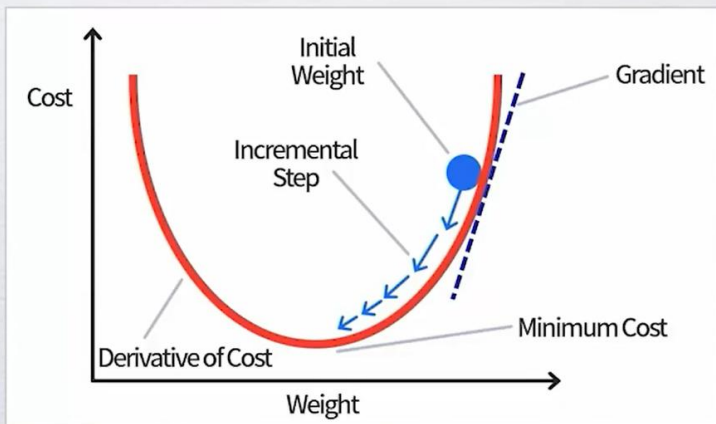
$$\text{MSE}(w) = \frac{1}{n} \sum_{i=1}^n (w^2 x_i^2 - 2wx_i y_i + y_i^2)$$

- w^2 계수가 항상 양수이므로 아래로 볼록한 U자형 곡선이 됨
- 최저점은 MSE가 최소가 되는 w 값(즉, 최적 파라미터)임
- 2차원 좌표평면에서는 x축에 w , y축에 MSE 값을 두면 그려짐

Cost 가장 낮아지는 수준까지 진행하면서 가중치 업데이트 진행

Gradient Descent Algorithm

그래프상에서 비용이 제일 낮은 아래부분(기울기 0)을 찾으면 된다!



- 최적의 w 를 찾기 위하여 **경사하강법**을 이용
- 임의의 w 로부터 시작하여 w 를 조금씩 감소시키면서 미분값(기울기)이 0이 되는 w 를 찾음
- 비용함수를 미분하면서 w 를 찾음

$$W := W - \alpha \frac{\partial}{\partial W} \text{cost}(W)$$

학습률
(w 를 감소시키는 정도)

$$\frac{1}{n} \sum_{i=1}^n (w^2 x_i^2 - 2w x_i y_i + y_i^2)$$

모델 학습이란?

목표: 최적 모델 구하기



모델별 손실함수 구하기



손실함수 최소값 구하기



Gradient Descent Algorithm 이용

$$y = wx$$

$$\text{MSE}(w) = \frac{1}{n} \sum_{i=1}^n (w^2 x_i^2 - 2wx_i y_i + y_i^2)$$

$$W := W - \alpha \frac{\partial}{\partial W} \text{cost}(W)$$

데이터 확보

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

잘 정리된 데이터 확보가 중요!

지도학습 (Supervised Learning)

- ▶ 정답을 알려주면서 진행하는 학습
- ▶ 데이터와 레이블(정답)이 함께 제공됨

레이블(Label) = 정답, 실제값, 타깃,
클래스, y

예측된 값 = 예측값, 분류값, $\hat{y}$ (y hat)

VS

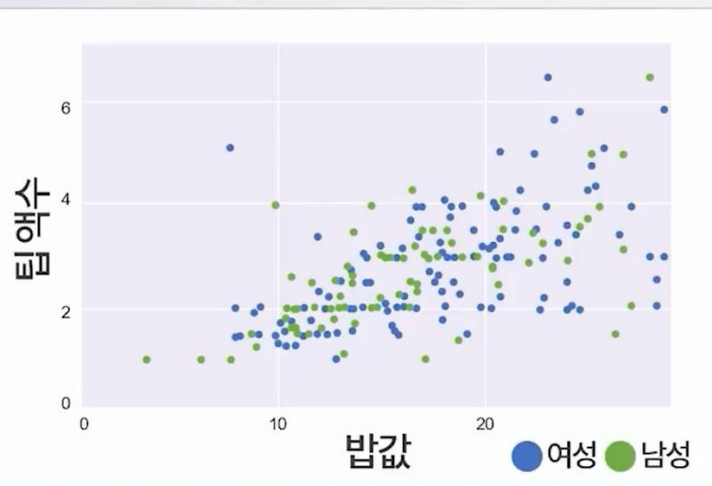
비지도학습 (Un-Supervised Learning)

- ▶ 레이블(정답) 없이 진행되는 학습
- ▶ 데이터 자체에서 패턴을 찾아내야 할 때
사용함

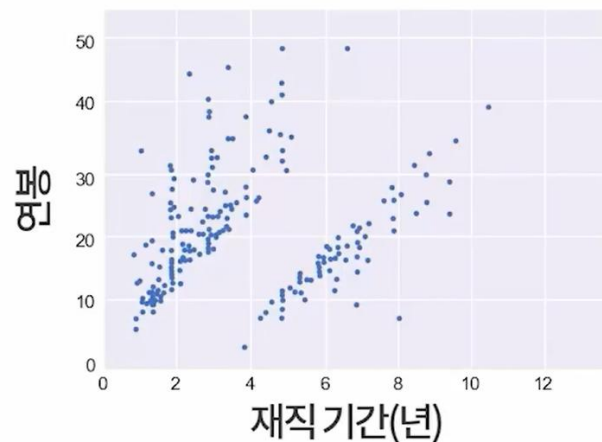
집 크기(m ²)	방 개수	실제 집값(정답, 원)
85	3	320,000,000
120	4	500,000,000
45	1	150,000,000
60	2	220,000,000

- 집 크기·방 개수 → 입력 데이터(Feature, X)
- 실제 집값 → 정답(Label, y)

지도학습 vs. 비지도학습



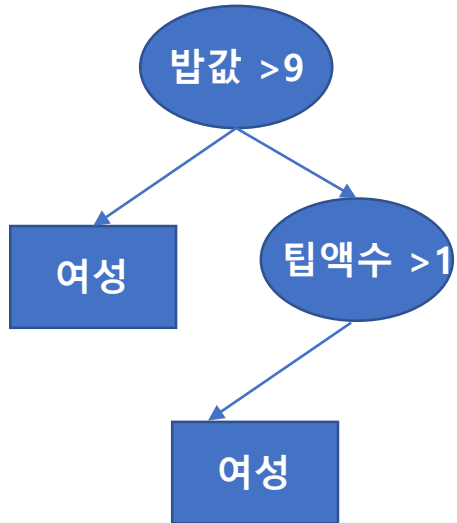
분류모델



군집분석

밥값	팁액수	성별(정답)
16.52	3.36	남성
9.80	0.18	여성
18.75	2.92	여성
19.70	3.31	여성
5.24	1.21	여성
8.49	1.30	여성

	재직기간(년)	연봉
1	1.92	36.86
2	1.47	12.79
3	1.87	50.46
4	1.78	18.25
5	2.83	29.77
6	2.47	15.54



지도학습 모델 종류

분류모델 (Classification)

레이블의 값들이 **이산적으로**
나뉘질 수 있는 문제에 사용

Example

밥값이 많이 나오면 남성,
적게 나오면 여성

예측 모델 (Regression, 회귀모델)

레이블의 값들이 **연속적인**
문제에 사용

Example

밥값이 많이 나올수록,
팁의 크기도 계속 커짐

$$\text{팁액수} = 2 * \text{밥값} + 3$$

지도학습 모델 종류

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

예측 모델(회귀 모델) 
팁의 크기를 예측

타깃 변수를

tip(숫자형)으로 설정하는 경우

분류 모델 

손님의 성별을 예측

타깃 변수를

sex(범주형)로 설정하는 경우

지도학습 데이터셋 구조

각 열(column)을 **특징/속성(feature)** 이라고 함

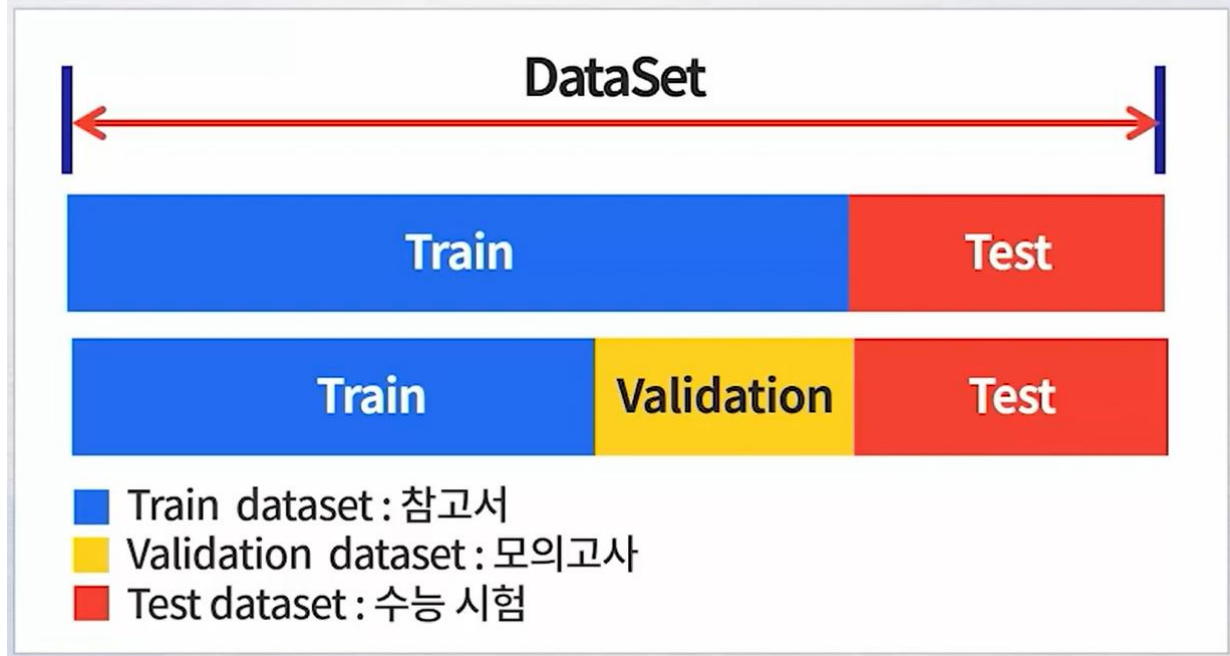
각 행(row)을 **예제(Example)** 데이터라고 함

	x_1	y	x_2	x_3	x_4	x_5	x_6
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

데이터 열(column) 중에 하나를 선택해서 **레이블**로 사용함

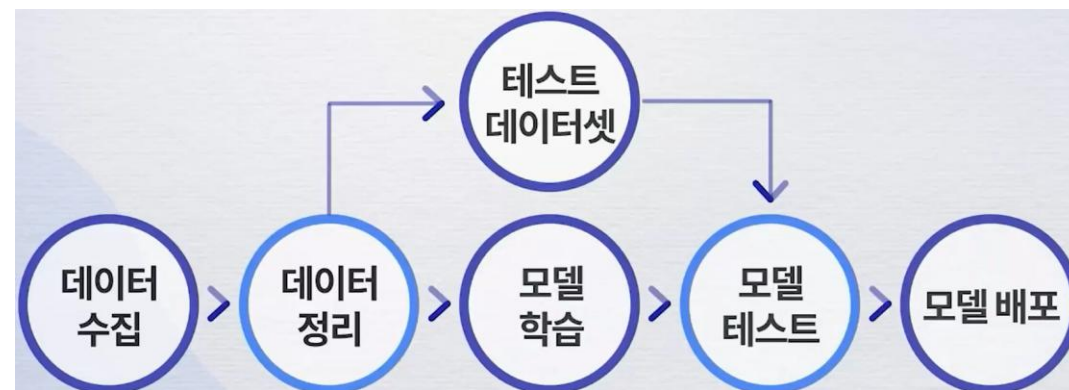
$$y = wx$$

데이터셋 분리



7:3

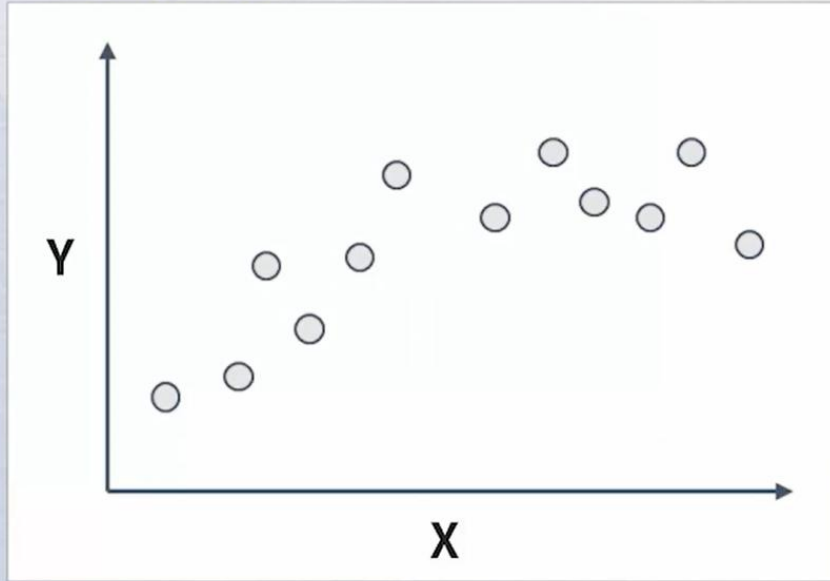
8:2



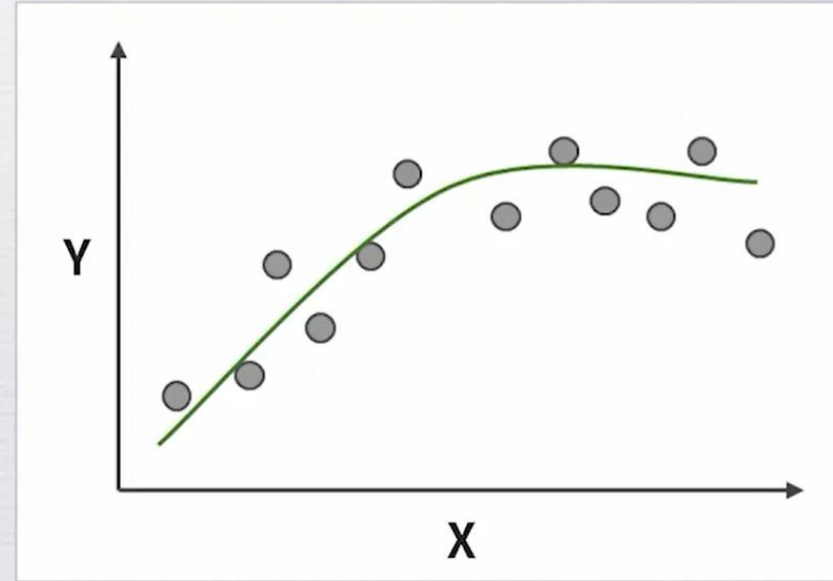
모델 선택



데이터



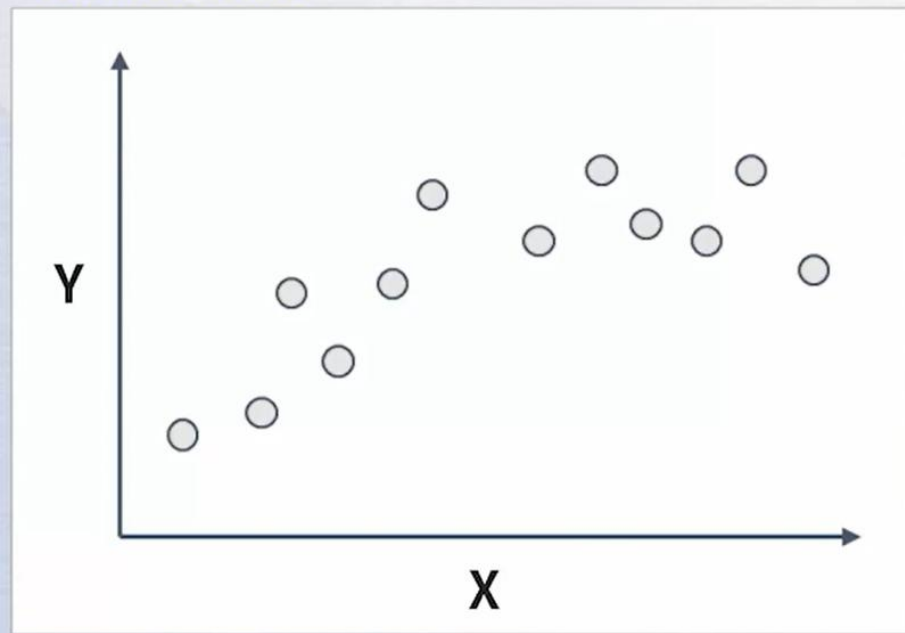
좋은 모델(Model)



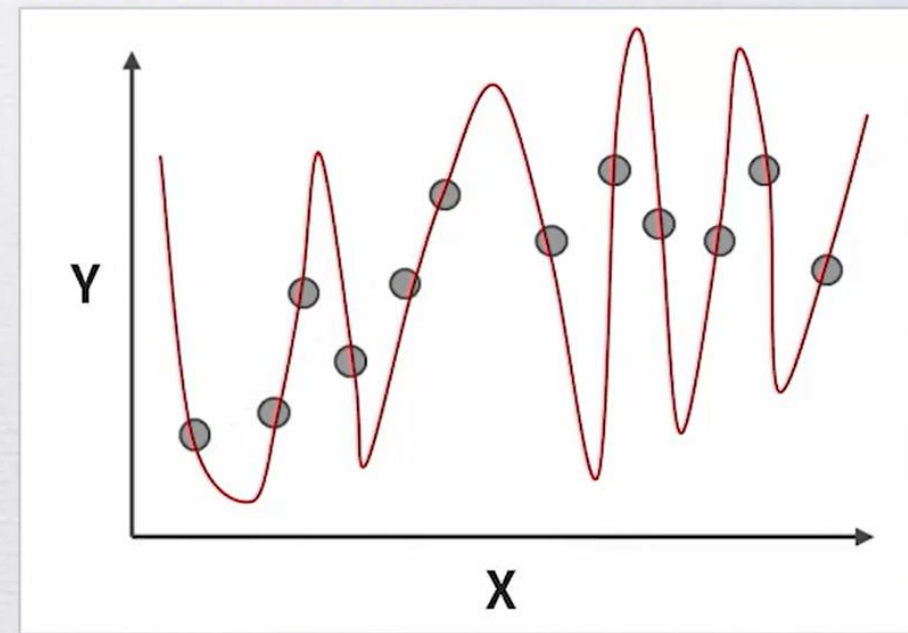
모델 선택



데이터



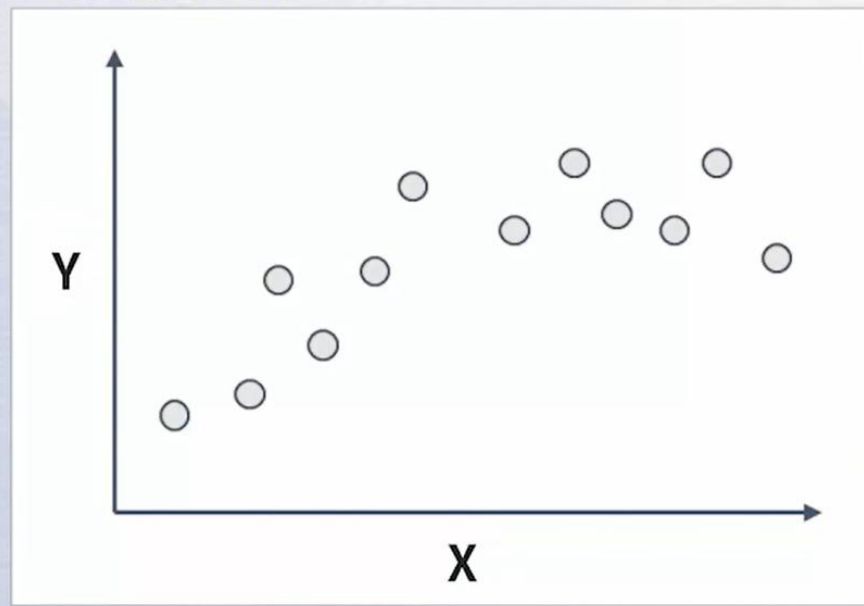
과적합(Overfitting)



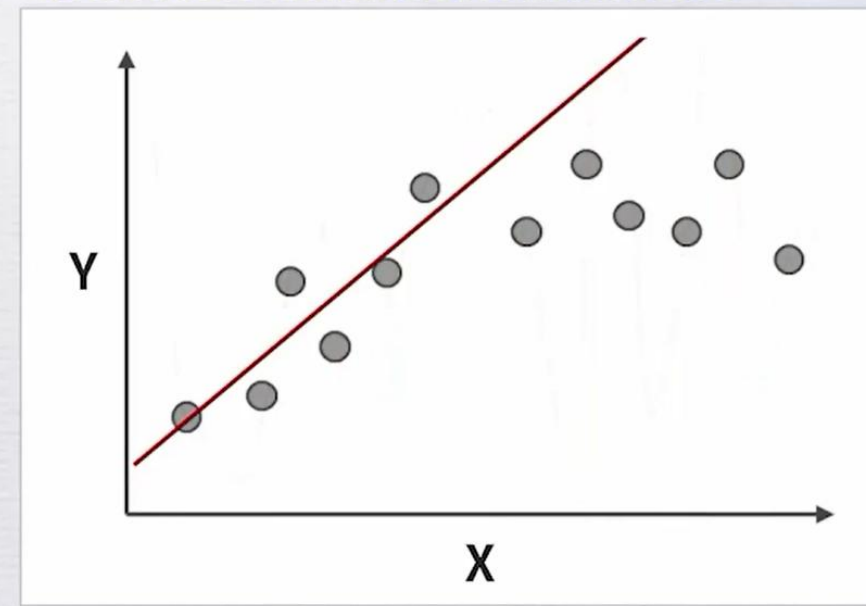
모델 선택



데이터

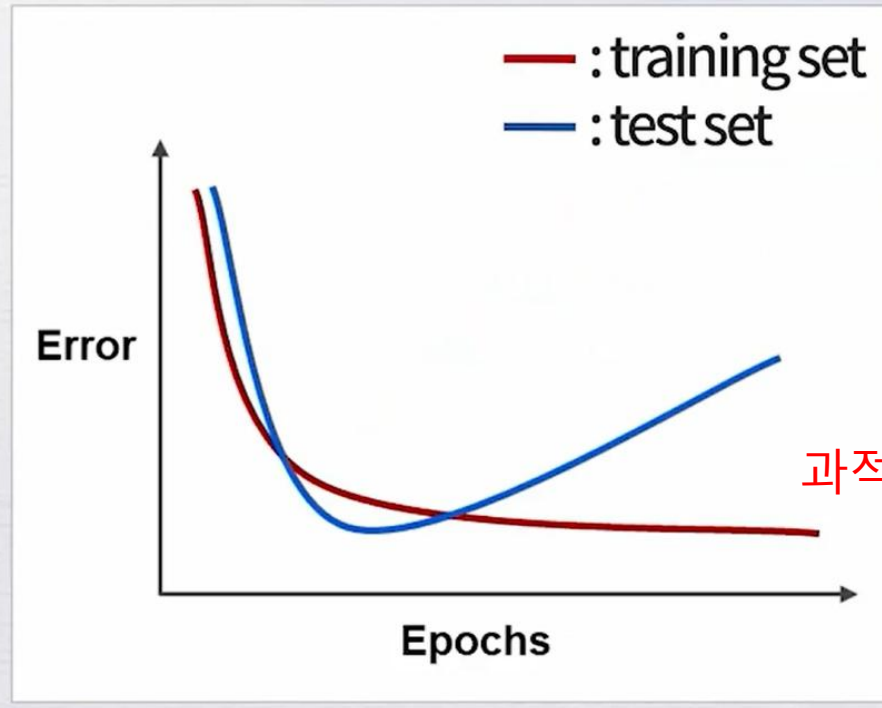
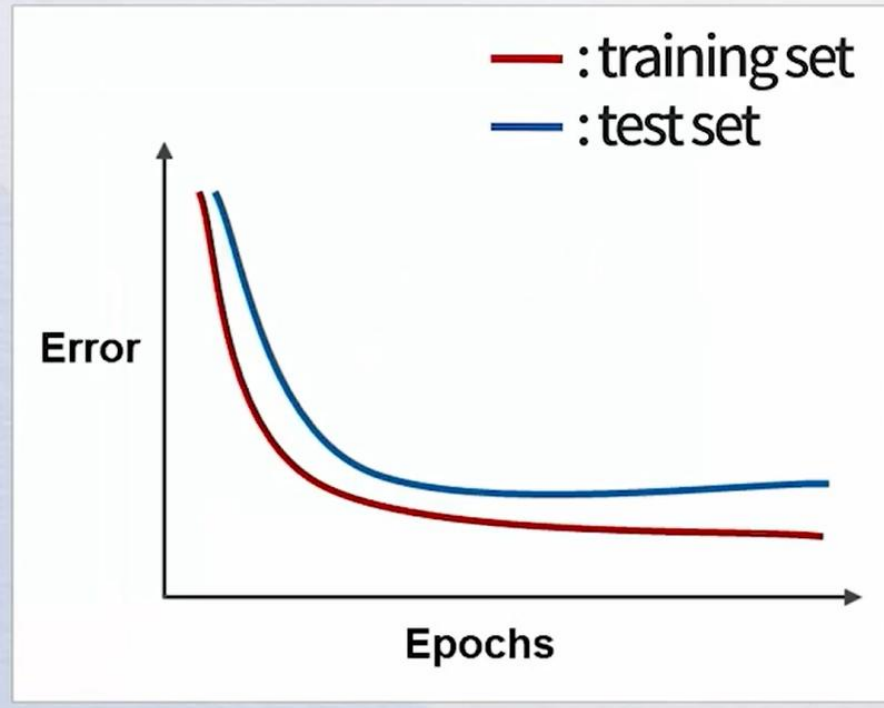


과소적합(Underfitting)



모델 성능 평가

$\text{error} = (\text{실제값} - \text{예측값})$
Epochs = 학습횟수



Confusion Matrix



모델의 대략적인 성능확인과 **모델의 성능**을
오차행렬을 기반으로 수치로 표현

Confusion Matrix

		예측값	
		0 (N)	1 (P)
실제값	0 (N)	TN	FP
	1 (P)	FN	TP

- ▶ True Positive(TP): True를 True로 예측 (정답)
- ▶ True Negative(TN): False를 False로 예측 (정답)
- ▶ False Positive(FP): False를 True로 예측 (오답)
- ▶ False Negative(FN): True를 False로 예측 (오답)

맞느냐 틀렸느냐 → **T N** ← 예측한 값

N(egative): 0

P(ositive): 1

Confusion Matrix

분류모델에 대한 성능지표 계산을 위한 데이터

	고객나이	성별	연봉	지역	실제응답여부	예측응답여부
1	25	여	3200	서울	1	1
2	40	남	5500	부산	0	0
3	31	여	4100	대전	1	0
4	22	남	2800	서울	0	0
5	35	여	5000	인천	1	1

	예측:미응답(0)	예측:응답(1)
실제:미응답(0)	2	0
실제:응답(1)	1	2

성능지표



학습이 끝난 후 **모델을 평가**하는 용도로 사용됨

Confusion Matrix

		예측값	
		0	1
실제값	0	TN	FP
	1	FN	TP

성능지표

정답으로 예측한 것 중 실제정답의 비율

실제 정답 중 제대로 예측한 것의 비율

정밀도(Precision)

재현율(Recall)

- ▶ 모델이 **True**라고 분류한 것 중에서 **실제 True**인 것의 비율
- ▶ 날씨 예측 모델이 맑다로 예측했는데, 실제 날씨가 맑았는지 나타낸 지표

- ▶ **실제 True**인 것 중에서 모델이 **True**라고 예측한 것의 비율
- ▶ 실제 날씨가 맑은 날 중에서 모델이 맑다고 예측한 비율을 나타낸 지표

$$(Precision) = \frac{TP}{TP + FP}$$

$$(Recall) = \frac{TP}{TP + FN}$$

암환자 진단은
재현율 중요

Confusion Matrix

		예측값	
		0	1
실제값	0	TN	FP
	1	FN	TP

예측한 것 중 제대로 맞춘 비율

정확도(Accuracy)

- ▶ 가장 직관적으로 모델의 성능을 나타낼 수 있는 평가 지표
- ▶ 한 달에 맑은 날이 28일이고 비가 오는 날이 2일인 경우, 비가 오는 것을 예측하는 성능은 매우 낮을 수 밖에 없으므로 이를 보완할 지표가 필요

$$(Accuracy) = \frac{TP + TN}{TP + FN + FP + TN}$$

성능지표

F1 점수(F1-score)

▶ 정밀도와 재현율의 조화평균

$$(F1-score) = 2 \times \frac{1}{\frac{1}{Precision} + \frac{1}{Recall}}$$

Precision과 Recall은 trade-off 관계이므로
Precision과 Recall을 모두 고려한 성능지표

머신러닝 기술원리(18/19)

모델 예측결과

	고객나이	성별	연봉	지역	실제응답여부	예측응답여부
1	25	여	3200	서울	1	1
2	40	남	5500	부산	0	0
3	31	여	4100	대전	1	0
4	22	남	2800	서울	0	0
5	35	여	5000	인천	1	1

Confusion Matrix

	예측:미응답(0)	예측:응답(1)
실제:미응답(0)	2 (TN)	0 (FP)
실제:응답(1)	1 (FN)	2 (TP)

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{2}{2 + 0} = 1.0$$

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{2}{2 + 1} \approx 0.6667$$

$$\text{Accuracy} = 4/5 = 0.8$$

정밀도

$$(\text{Precision}) = \frac{TP}{TP + FP}$$

재현율

$$(\text{Recall}) = \frac{TP}{TP + FN}$$

$$(\text{Accuracy}) = \frac{TP + TN}{TP + FN + FP + TN}$$

정확도

머신러닝의 주요 알고리즘(1/26)

◆ 머신러닝 알고리즘

- 데이터로부터 패턴을 학습해 예측이나 분류를 할 수 있게 해주는 '절차(방법)
- **모델을 만드는 방법**을 정의한 규칙과 단계의 집합
- 요리 레시피: 재료(데이터)를 어떻게 손질하고 조리하면 원하는 음식(모델)을 만들 수 있는지 순서와 방법이 적혀 있음
- 머신러닝 알고리즘: 데이터로 어떻게 학습을 진행해서 최종 모델을 만들지, 수학적·논리적 절차가 적혀 있음

◆ 머신러닝 알고리즘의 일반적 구조

1. 입력 데이터(X): 특성(feature)
2. 학습 규칙: 가중치 업데이트, 오차 계산, 반복 방법
3. 출력: 학습된 모델(예측 가능)



머신러닝의 주요 알고리즘(2/26)

◆ 대표적인 머신러닝 알고리즘

구분	알고리즘	설명	도출될 수 있는 모델명	모델 응용 예
지도학습	선형회귀 (Linear Regression)	연속값 예측	선형회귀모델	주택가격예측모델, 매출예측모델
지도학습	로지스틱회귀 (Logistic Regression)	범주형 분류	로지스틱회귀모델	스팸메일분류모델, 질병진단모델
지도학습	의사결정나무 (Decision Tree)	규칙 기반 분류·예측	의사결정나무모델	고객이탈예측모델, 대출승인판정 모델
지도학습	서포트 벡터 머신 (SVM)	초평면으로 구분	서포트벡터머신모델	얼굴인식모델, 필기체인식모델
비지도학습	K-평균 (K-Means)	데이터 군집화	K-평균군집모델	고객세분화모델, 상품추천군집모델
비지도학습	PCA	차원 축소	PCA차원축소모델	이미지압축모델, 특징추출모델

◆머신러닝 알고리즘을 구현한 파이썬 라이브러리

사이킷런

The Scikit-learn logo, featuring the text "Scikit-learn" in a blue, sans-serif font. Below the text is a small horizontal bar with a yellow-to-blue gradient.

가장 인기 있는 머신러닝 패키지,
많은 머신러닝 알고리즘이 내장되어 있음

파이썬 코딩에서 머신러닝 기반 학습시킬 때 필요

◆사이킷런으로 회귀모델을 생성하는 예

문법 형식

Scikit-learn

```
from sklearn.family import Model
```



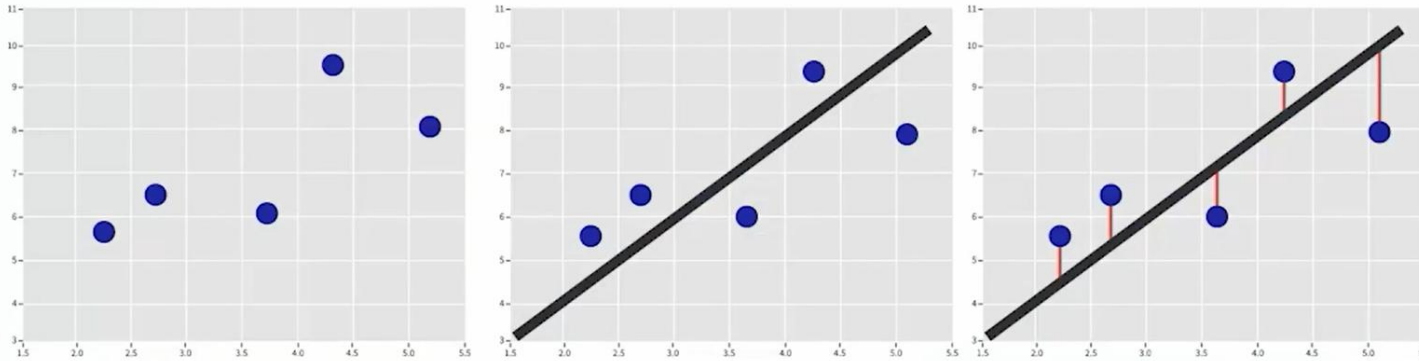
```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()  
print(model)
```



Linear Regression

데이터의 분포 패턴에 가장 적합한 직선을 찾아서 예측

$$y = wx$$



Linear Regression

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
```

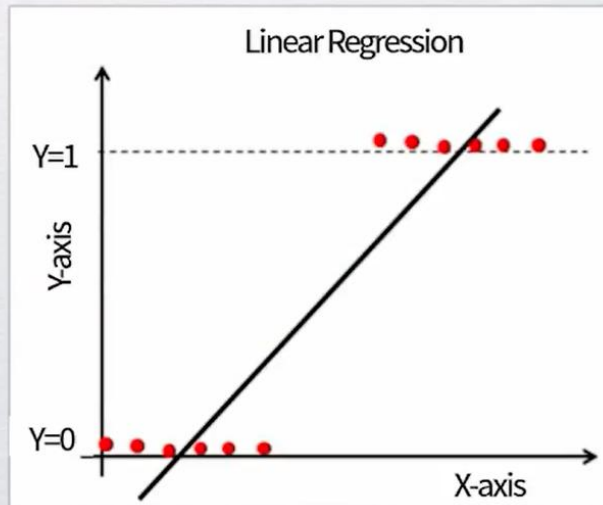
```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

1. 입력 데이터(X): 특성(feature)
2. 학습 규칙: 가중치 업데이트, 오차 계산, 반복 방법
3. 출력: 학습된 모델(예측 가능)

Logistic Regression

이진 분류 규칙은 0과 1의 두 클래스를 갖는 것으로,
일반 선형 회귀 모델을 이진분류에 사용하기 어려움



$$y = wx$$

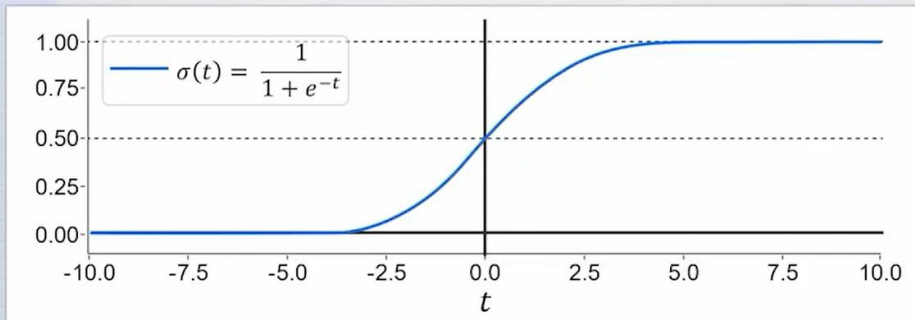
과소적합

출처: <https://ichi.pro/ko/lojiseutig-hoegwi-ihae-mich-python-eseo-model-guchug-25643771585192>

Logistic Regression

로지스틱(logistic) 함수를 사용하여 로지스틱 회귀 곡선으로
변환하여 이진 분류 할 수 있음

✓ 로지스틱 함수



$$y = wx$$

$$\hat{y} = \begin{cases} 0 & \text{if } y < 0.5 \\ 1 & \text{if } y \geq 0.5 \end{cases}$$

시그모이드(=로지스틱) 함수:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- 입력 z 를 0~1 사이의 확률 값으로 변환
- S자(S-shape) 곡선을 가짐

$$y = wx$$

과소적합

$$\hat{y} = \frac{1}{1 + e^{-wx}}$$

최적적합

로지스틱 회귀모델
(데이터에 따라, w 가 달라질 것임)

Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

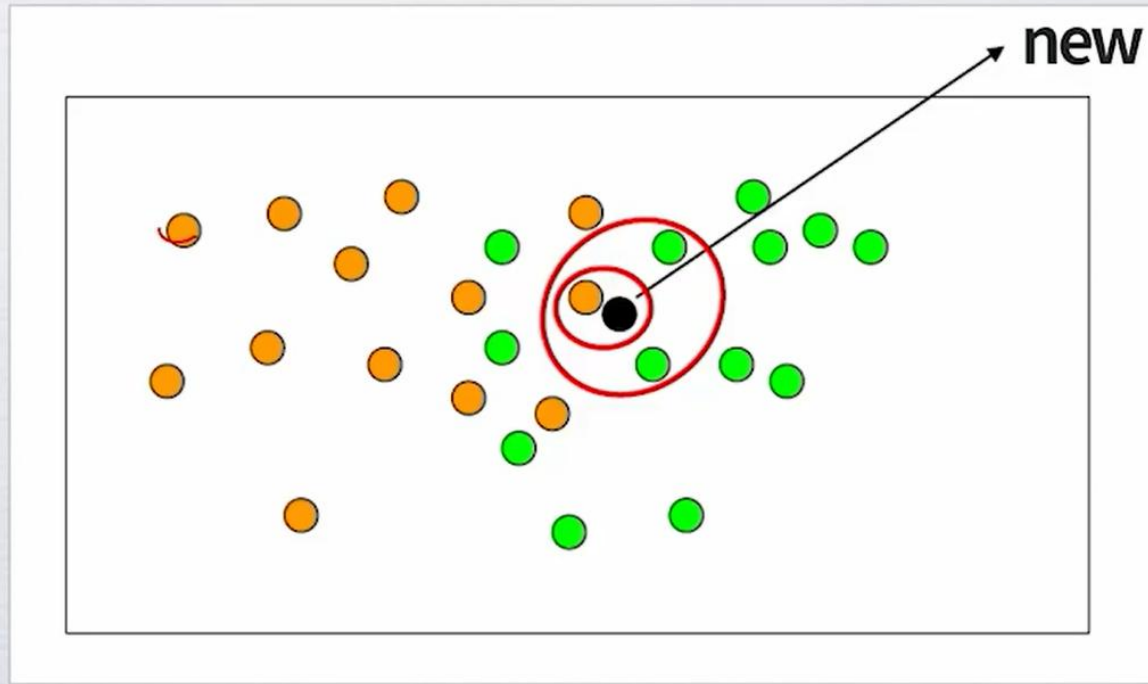
```
pred = model.predict(X_test)
```

1. 입력 데이터(X): 특성(feature)
2. 학습 규칙: 가중치 업데이트, 오차 계산, 반복 방법
3. 출력: 학습된 모델(예측 가능)

K-Nearest Neighbor

새로운 데이터가 주어졌을 때 기존 데이터 가운데
가장 가까운 k 개 이웃의 정보로 새로운 데이터를 예측하는 방법론

K-Nearest Neighbor



k 파라미터 역할 : 이웃의 갯수 설정

분류는 이웃 노드의 다수 class로 분류
예측은 이웃 노드의 평균값으로 예측

k = 1 : 검은점은 누런색으로 분류됨

k = 3 : 검은점은 연두색으로 분류됨

K-Nearest Neighbor

알고리즘이 간단하며
큰 데이터셋과 고차원 데이터에 적합하지 않음

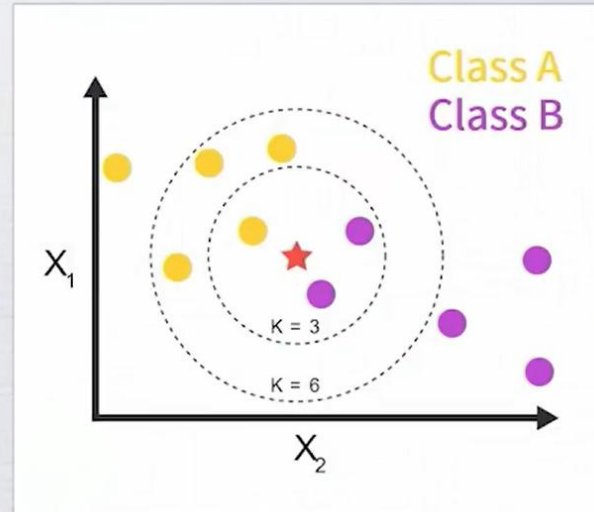
K-Nearest Neighbor

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=3)
```

```
knn.fit(X_train, y_train)
```

```
pred = knn.predict(X_test)
```



Decision Tree

분류와 회귀 작업이 가능한 다재다능한 머신러닝 알고리즘



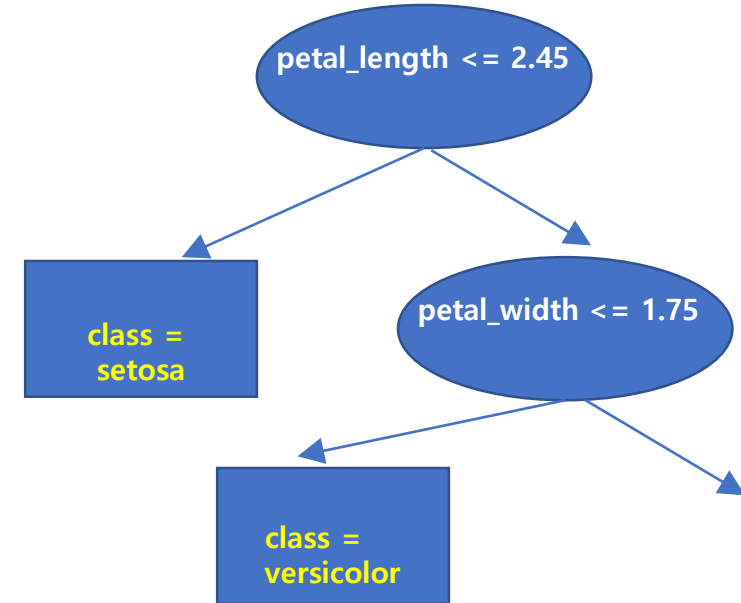
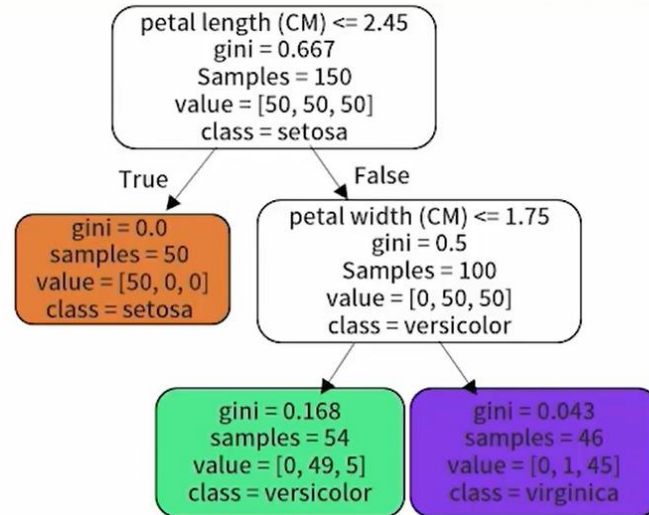
복잡한 데이터셋도 학습할 수 있으며
강력한 머신러닝 알고리즘인 **랜덤 포레스트**의 기본 구성 요소

Decision Tree

붓꽃(Iris)



붓꽃 분류 결정트리



Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
```

```
model = DecisionTreeClassifier(max_depth=2)
```

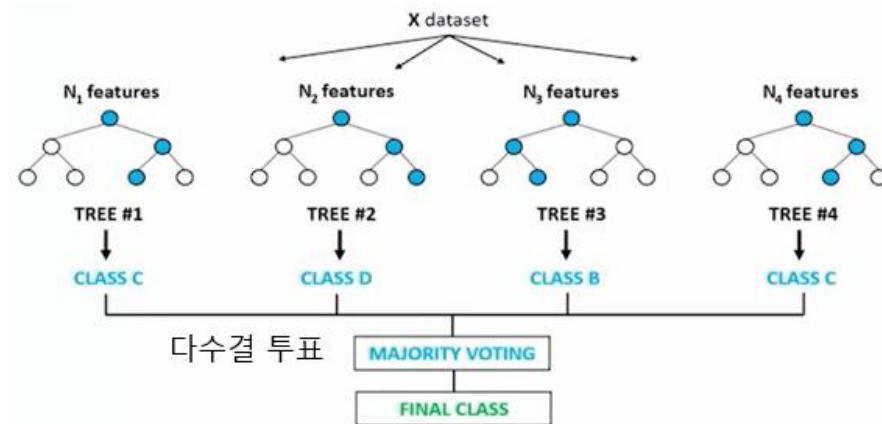
```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

머신러닝의 주요 알고리즘(18/26)

1. 앙상블 방법 (Ensemble Method)

- 여러 개의 모델을 결합해 하나의 최종 예측을 만드는 방법
- 목적: 개별 모델보다 정확도·안정성 향상
- 대표 유형
 - Bagging** (Bootstrap Aggregating) → 모델을 병렬로 학습, 예측 결과 평균 또는 투표
 - 예: 랜덤 포레스트
 - Boosting** → 이전 모델의 오류를 보완하는 새로운 모델을 순차적으로 학습
 - 예: XGBoost, AdaBoost, LightGBM
 - Stacking** → 여러 모델의 예측을 다시 학습해서 최종 예측

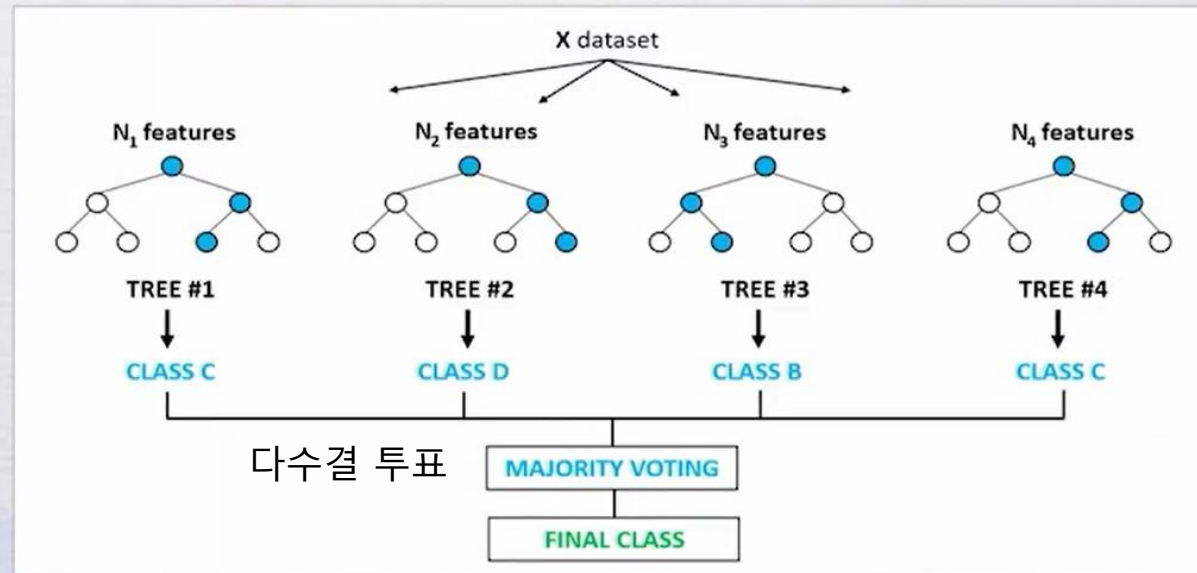


2. 랜덤 포레스트 (Random Forest)

- 앙상블 방법 중 Bagging 계열
- 의사결정나무(Decision Tree) 여러 개를 만들어서 투표/평균으로 최종 예측
- 핵심 아이디어:
 - 학습 데이터 샘플링(부트스트랩 샘플)
 - 각 트리마다 특성을 무작위로 선택하여 다양성 확보

Random Forest

훈련 세트로부터 무작위로 각기 **다른 서브셋**을 만들어
일련의 결정 트리 분류기를 훈련시킬 수 있음



출처:
<https://www.kaggle.com/getting-started/176257>

Random Forest

```
from sklearn.ensemble import RandomForestClassifier
```

```
model = RandomForestClassifier(n_estimators=50)
```

```
model.fit(X_train, y_train)
```

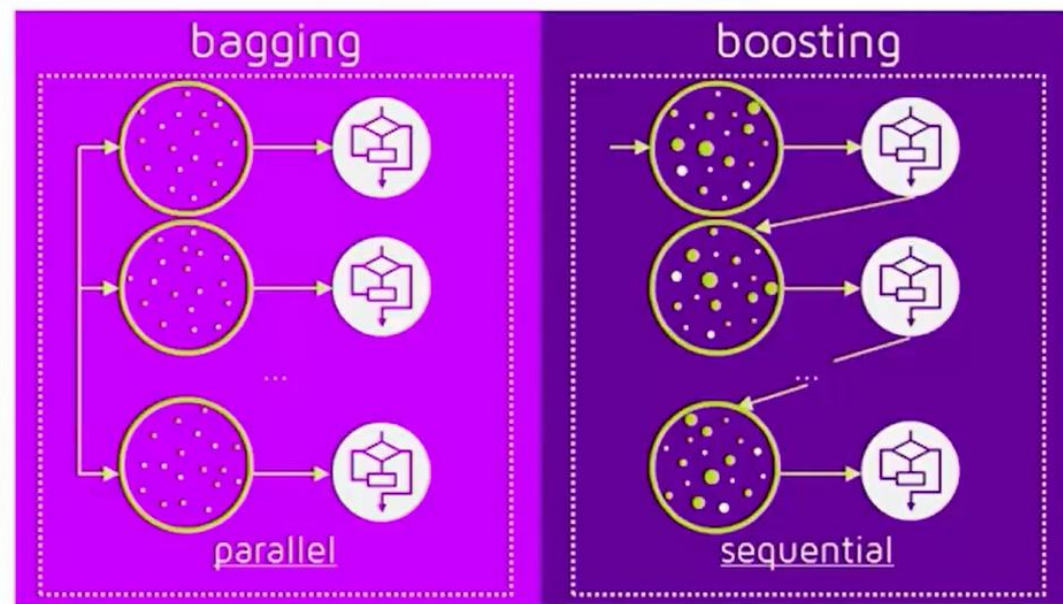
```
pred = model.predict(X_test)
```

Ensemble 기법

1) Boosting

Boosting → 이전 모델의 오류를 보완하는 새로운 모델을 순차적으로 학습

- 예: XGBoost, AdaBoost, LightGBM



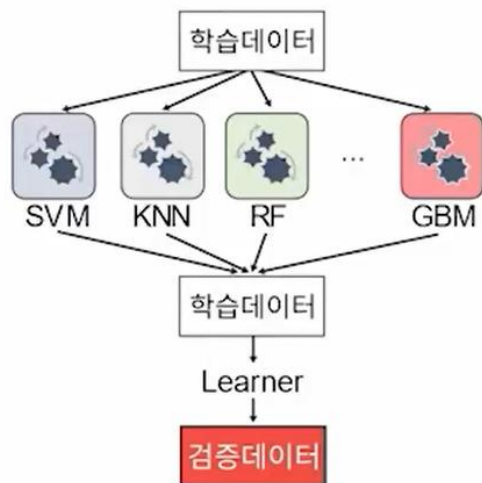
부스팅(Boosting) 은 여러 약한 모델(weak learners)을 순차적으로 학습시켜서, 결국 하나의 **강한 모델(strong learner)** 을 만드는 방법

출처:

<https://medium.com/swlh/difference-between-bagging-and-boosting-f996253acd22>

Ensemble 기법

2) Stacking



스태킹(Stacking)에서 "여러 모델의 예측을 다시 학습한다" 는 말은, 단순히 각 모델의 결과를 평균 내는 게 아니라, 그 결과값(예측값)을 새로운 학습 데이터로 사용해서 또 다른 모델을 훈련.

1단계(기본 학습기, Base Learners)

- 서로 다른 여러 모델을 학습시킴
예: 로지스틱 회귀, 랜덤포레스트, SVM, XGBoost ...
- 각 모델이 예측값을 출력함

2단계(메타 학습기, Meta Learner)

- 1단계 모델들이 낸 예측값들을 입력 데이터(feature) 로 사용
- 이 새로운 데이터와 정답 라벨을 이용해 또 하나의 모델을 학습
- 이 모델이 최종 예측을 수행

Ensemble 기법

3) Weighted Blending

▶ 각 모델의 예측값에 대하여 weight를 곱하여 최종 output 계산



가중치의 합은 **1.0**이 되도록 함

KNN(가중치 0.25)

RandomForest(가중치 0.5)

Logistic Regression(가중치 0.25)

Ensemble 대표 모델

1) XGBoost

```
!pip install xgboost  
from xgboost import XGBClassifier
```

```
model = XGBClassifier(n_estimators=50)
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

Ensemble 대표 모델

2) LightGBM

```
!pip install lightgbm  
from xgboost import LGBMClassifier
```

```
model = LGBMClassifier(n_estimators=50)
```

```
model.fit(X_train, y_train)
```

```
pred = model.predict(X_test)
```

머신러닝의 주요 알고리즘(26/26)

기본 개념

- ▶ 최적의 선을 긋는 것
- ▶ Cost Function
- ▶ Gradient Decent
(경사하강법)

기술 원리

- ▶ 지도학습 vs 비지도학습
- ▶ 회귀와 분류
- ▶ 데이터 확보/전처리 시간
소요
- ▶ 모델 성능 지표

주요 알고리즘

Linear Regression

Logistic Regression

Decision Tree

LightGBM

XGBoost

Random Forest

파이썬 머신러닝 실습

```
(4)-머신러닝.py x
1  """
2  #####
3  [실습-정답] Python을 활용한 AI 모델링 - 머신러닝 파트
4  #####
5
6  - 이번시간에는 Python을 활용한 AI 모델링에서 머신러닝에 대해 실습해 보겠습니다.
7  - 머신러닝 모델에는 아래와 같이 모델들이 있습니다.
8      => 단일 분류예측 모델 : LogisticRegression, KNN, DecisionTree
9      => 앙상블(Ensemble) 모델 : RandomForest, XGBoost, LGBM, Stacking, Weighted Blending
10 - 솔직히, 머신러닝이 딥러닝보다 코딩하기 쉽습니다. 4줄 템플릿에 맞춰 코딩하면 되기 때문입니다.
11 - 한가지 당부 드리고 싶은 말은 "백문이불여일타" 입니다.
12 - 이보다 실습이 더 많은 시간과 노력이 투자 되어야 합니다.
13
14 *****
15 학습목차
16 *****
17
18
19 1. 머신러닝 모델 프로세스
20 - 데이터 가져오기
21 - 데이터 전처리
22 - Train, Test 데이터셋 분할
23 - 데이터 정규화
24 - 단일 분류예측 모델 : LogisticRegression, KNN, DecisionTree
25 - 앙상블(Ensemble) 모델 : RandomForest, XGBoost, LGBM, Stacking, Weighted Blending
26
27 """
28
29 ## 1. 실습준비
30 """
31 # 코드실행시 경고 메시지 무시
32
33 import warnings
34 warnings.filterwarnings(action='ignore')
35 """
36
37 """
38 #####
39 2. 머신러닝 모델 프로세스
40 #####
```

감사합니다.